



EL2805 Reinforcement Learning

Exercise Session 4

November 25th 2025

Division of Decision and Control Systems
School of Electrical Engineering and Computer Science
KTH Royal Institute of Technology

Instructions.

- These exercises are for your practice, and you do not need to submit solutions. We encourage you to work on them before the corresponding exercise session on Nov 28th. The materials for the 5th exercise session will be available on Nov 28th.
- In each exercise session, we will go through a selection of topics chosen by the teaching assistant and participating students.
- Homework and labs will be based on concepts covered in lectures and exercise sessions. These materials are excellent preparation for your graded assignments!
- There is no need to memorize equations - relevant materials will be provided during the exam. Focus on concepts and understanding!
- The materials cover a wide range of topics, so feel free to focus on what interests you most. However, to develop a well-rounded understanding of RL, it is important that you engage with both theoretical and practical exercises.
- We use green boxes to highlight established results and definitions, yellow for problems that require your solutions, and pink for high-level remarks. Exercises marked with a ★ are more challenging and intended for students seeking an extra challenge.
- We value your feedback on the materials provided. If you have any questions, concerns, or suggestions, do not hesitate to reach out to any of the TAs.

1 Intro to policy gradient methods with REINFORCE

In this exercise, we will explore policy gradient algorithms. The material follows Lectures 6 and 8, while other relevant materials are e.g. Sections 10 and 13 in [1], or online resources such as [2].

Until now, we relied on dynamic programming-style algorithms, which help us estimate the value function of a policy, improve the policy, and iterate this procedure. One may ask: instead of estimating value functions, can we directly optimize over the space of all possible policies Π ? Unfortunately, this is intractable in general. However, if we parametrize a family of policies π_θ , we can optimize over the parameter space instead:

$$\max_{\theta} J(\theta) = \left[\max_{\theta} \mathbb{E}_{s_1 \sim \rho} [V^{\pi_\theta}(s_1)] \right] = \max_{\theta} \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{u=1}^T r(s_u, a_u) \right]$$

Here, the trajectory $\tau = (s_1, a_1, r_1, s_2, a_2, r_2, \dots, r_T)$ is sampled under the policy π_θ . The next step is to compute the gradient $\nabla_{\theta} J(\theta)$. Using the log-derivative trick, we obtain:

$$\nabla_{\theta} \left[\sum_{\tau} \pi_{\theta}(\tau) \sum_{u=1}^T r(s_u, a_u) \right] = \sum_{\tau} \pi_{\theta}(\tau) \underbrace{\frac{\nabla_{\theta} \pi_{\theta}(\tau)}{\pi_{\theta}(\tau)}}_{\nabla \log \pi_{\theta}(\tau)} \sum_{u=1}^T r(s_u, a_u) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\nabla_{\theta} \log \pi_{\theta}(\tau) \sum_{u=1}^T r(s_u, a_u) \right]$$

Using the Markov property and the factorization of trajectory probabilities:

$\pi_{\theta}(\tau) = \rho(s_1) \pi_{\theta}(a_1|s_1) P(s_2|s_1, a_1) \pi_{\theta}(a_2|s_1) \dots P(s_T|s_{T-1}, a_{T-1}) \pi_{\theta}(a_T|s_T)$, we get:

$$\nabla_{\theta} \log \pi_{\theta}(\tau) = \nabla_{\theta} \log \left(\rho(s_1) \pi_{\theta}(a_1|s_1) \prod_{k=2}^T P(s_k|s_{k-1}, a_{k-1}) \pi_{\theta}(a_k|s_k) \right) = \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$$

This leads to the REINFORCE update rule: each parameter update is proportional to the total return multiplied by a vector that increases the likelihood of the actions actually taken.

Algorithm 1 REINFORCE

- 1: **Initialization:** select initial policy parameters $\theta^{(0)}$, learning rates $\{\alpha_k\}_{k \in \mathbb{N}}$
- 2: **for** each episode $k = 1, 2, \dots$ **do**
- 3: Generate N trajectories under $\pi_{\theta^{(k)}}$: $\tau_i^{(k)} = (s_{1,i}, a_{1,i}, r_{1,i}, \dots, s_{T,i}, a_{T,i}, r_{T,i})$, $i \in [N]$.
- 4: Update the parameter θ :

$$\theta^{(k+1)} = \theta^{(k)} + \alpha_k \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{t,i}|s_{t,i}) \right) \left(\sum_{u=1}^T r_{u,i} \right)$$

- 5: **end for**
-

1.1 [solve] Policy Gradients for a Winning Strategy

We now illustrate policy gradients and REINFORCE with a simple example:

Exercise 1.1 (Rock-Paper-Scissors game). You play against an opponent who selects actions in an i.i.d. manner, according to a known distribution $\mu = (\mu_R, \mu_S, \mu_P)$.

- a) Model this as an MDP with $\mathcal{S} = \{\text{Play, Won, Lost}\}$. What is the optimal policy?
- b) Now assume the opponent instead follows a policy such that each time they play a move, the probability of repeating the same move decreases by one third, and their first action is drawn from μ . How would the model and optimal policy change?

Knowing the distribution of your opponent's actions is a strong assumption. In practice, we often do not know how our opponents choose their actions. Next, we study how REINFORCE can help find an optimal policy when this distribution is unknown.

Exercise 1.2 Assume now that the opponent's distribution μ is **unknown**, but fixed.

- Propose a policy parametrization (use softmax with linear action preferences).
- Explain how to apply REINFORCE to learn an optimal policy and derive gradients.

1.2 [solve] Reducing Gradient Variance in REINFORCE

REINFORCE, as introduced in the previous section, suffers from high variance in its gradient estimates. We now study two techniques to reduce the variance.

Technique 1: Causality Principle. We replace $\nabla_{\theta} J(\theta)$ with an equivalent expression:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \sum_{u=1}^T r(s_u, a_u) \right] \rightarrow \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \underbrace{\sum_{u=t}^T r(s_u, a_u)}_{\text{reward to go}} \right]$$

Intuitively, an action taken at time step t can only influence future rewards $r_u = r(s_u, a_u)$ for $u \geq t$. It cannot influence rewards that have already been observed. By summing only over relevant contributions, we reduce the variance of the estimates. Next we show that this new estimator is unbiased.

Exercise 1.3 First show that $\sum_a \pi_{\theta}(a|s) \nabla_{\theta} \log \pi_{\theta}(a|s) = 0$. Then, using this equation and Markovian property, show the *causality principle*:

$$\mathbb{E}_{\tau \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) r(s_u, a_u)] = 0 \quad \text{for } u < t.$$

The above expression corresponds to the terms omitted in our gradient redefinition. Since they are all zero, the two expressions are equivalent in expectation, but the proposed estimate has lower variance.

Technique 2: Baseline Subtraction. Consider an MDP with high rewards (e.g., ≈ 1000). Learning small improvements (e.g., preferring 1001 over 1000) would require many samples. Instead, we can subtract a baseline value from the rewards. We do this by rewriting the gradient $\nabla_{\theta} J(\theta)$ as follows:

$$\nabla_{\theta} J(\theta) \rightarrow \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \left(\sum_{u=t}^T r(s_u, a_u) - b(s_t) \right) \right]$$

Using reasoning similar to the previous exercise, one can show that adding a baseline does not change the gradients (try to show it!).

1.3 [code] REINFORCE your Understanding

Take some time to implement REINFORCE and verify your understanding of the theoretical concepts correctly. Policy gradient methods are fundamental to almost all state-of-the-art RL methods, so ensuring a solid grasp of the basics is essential.

Exercise 1.4 Implement REINFORCE for CartPole. Apply the reward-to-go and baseline variance-reduction techniques, and analyze their impact on training. Further instructions can be found in the script `CartPole_REINFORCE.py`.

2 Intro to trust region methods with TRPO

“Never take a step longer than your leg.”

– A. Kraus (Spanish tenor)

In this section, you will explore one of the most popular RL algorithms: Trust Region Policy Optimization (TRPO) [3]. Given a trajectory τ , we focus on the discounted return $J(\pi)$:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r(s_t, a_t) \right]$$

Since this is infinite-horizon MDP, we cannot directly apply REINFORCE, as episodes do not terminate. Instead, we use the advantage function to quantify the quality of the current policy. Recall that the advantage function is defined on state-action pairs as $A_{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$. At each iteration of a policy gradient algorithm, we **update the policy** π' (or $\pi_{\theta(k+1)}$) using the **previous policy** π (or $\pi_{\theta(k)}$) and its advantage function A_{π} .

2.1 Deriving the TRPO objective function

Exercise 2.1 Show that for every π, π' : $J(\pi') = J(\pi) + \mathbb{E}_{\tau \sim \pi'} \left[\sum_{t=1}^{\infty} \gamma^{t-1} A_{\pi}(s_t, a_t) \right]$.

△Hints

1. From the definitions, show that: $A_{\pi}(s_t, a_t) = \mathbb{E}_{(s_{t+1}|s_t, a_t)} [r(s_t, a_t) + \gamma V^{\pi}(s_{t+1}) - V^{\pi}(s_t)]$.
2. Rewrite each term in $\mathbb{E}_{\tau \sim \pi'} \left[\sum_{t=1}^{\infty} \gamma^{t-1} A_{\pi}(s_t, a_t) \right]$ using the result from 1, and cancel out as many $V^{\pi}(s_t)$ terms as possible.
3. Recall that $J(\pi) = \mathbb{E}_{s_1 \sim \rho} V^{\pi}(s_1)$ for the initial state distribution ρ .

Next, recall the discounted occupancy measure d^{π} from our first session (exercise 2.2 there): $d^{\pi}(s) = (1 - \gamma) \sum_{t=1}^{\infty} \gamma^{t-1} \mathbb{P}_{\pi}(s_t = s)$. We can now express $J(\pi')$ as an expectation over $A_{\pi}(s, a)$:

$$\begin{aligned} J(\pi') &= J(\pi) + \sum_{t=1}^{\infty} \gamma^{t-1} \sum_{s,a} \mathbb{P}_{\pi'}(s_t = s) \pi'(a|s) A_{\pi}(s, a) \\ &= J(\pi) + \sum_s \sum_{t=1}^{\infty} \gamma^{t-1} \mathbb{P}_{\pi'}(s_t = s) \sum_a \pi'(a|s) A_{\pi}(s, a) = J(\pi) + \frac{1}{1 - \gamma} \mathbb{E}_{\substack{s \sim d^{\pi'}(s) \\ a \sim \pi'(a|s)}} A_{\pi}(s, a) \end{aligned}$$

Since π' is our next policy (that we want to find!), we do not yet have states sampled from it. Instead, we can only work with states sampled from the current policy i.e., $s \sim d^{\pi}$. This motivates defining the following surrogate objective:

$$L_{\pi}(\pi') = J(\pi) + \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^{\pi}(s), a \sim \pi'(a|s)} A_{\pi}(s, a)$$

Thus, $L_{\pi}(\pi')$ is a quantity we can compute from collected data, and we aim to maximize it as a proxy for $J(\pi')$. At a high level, much like a Taylor expansion provides a local approximation for a function, $L_{\pi}(\pi')$ represents a local approximation of $J(\pi')$. TRPO uses this principle to define **trust regions**, ensuring that policy updates stay within regions where this approximation is valid.

2.2 How big is the trust region for the surrogate objective?

As in real analysis, approximation errors depends on the distance between points. Similarly, the quality of our estimate depends on the distance between the distributions d^{π} and $d^{\pi'}$. Distances between distributions can be measured using the max- Kullback-Leibler (KL) divergence and Total-Variation (TV) distance:

$$D_{\text{KL}}^{\max}(\pi, \pi') = \max_s \sum_a \pi(a|s) \log \frac{\pi(a|s)}{\pi'(a|s)}, \quad D_{\text{TV}}^{\max}(\pi, \pi') = \frac{1}{2} \max_s \sum_a |\pi(a|s) - \pi'(a|s)|$$

The following intermediate results are used in the main derivation (Exercise 2.3). Feel free to skip solving the next exercise the first time you go through this material.

★ **Exercise 2.2** Let $C = \max_{s,a} |A_\pi(s, a)|$. Show the following two inequalities:

$$a) \max_s |\mathbb{E}_{a \sim \pi'(\cdot|s)} A_\pi(s, a)| \leq 2CD_{\text{TV}}^{\max}(\pi, \pi'), \quad b) \sum_s |d^\pi(s) - d^{\pi'}(s)| \leq \frac{2\gamma}{1-\gamma} D_{\text{TV}}^{\max}(\pi, \pi').$$

△Hints

To show b), rewrite $d^\pi - d^{\pi'}$ using $d^\pi = (1-\gamma) \sum_{t=1}^{\infty} \gamma^{t-1} \rho P_\pi^{t-1}$ as telescoping sums.

So far, we have established that maximizing $L_\pi(\pi')$ is simpler than maximizing $J(\pi')$. Next, we show that there exists a function $\delta(\pi, \pi') > 0$ such that $J(\pi') \geq L_\pi(\pi') - \delta(\pi, \pi')$. If we maximize $L_\pi(\pi')$ such that $L_\pi(\pi') \geq L_\pi(\pi) = J(\pi)$, then:

$$J(\pi') \geq L_\pi(\pi') - \delta(\pi, \pi') \geq J(\pi) - \delta(\pi, \pi')$$

By ensuring that $\delta(\pi, \pi') \approx 0$ is small (i.e., taking small steps), we guarantee that $J(\pi') \geq J(\pi)$, thereby achieving policy improvement.

Exercise 2.3 (Main result) Show that the two objectives are close with distance at most:

$$|J(\pi') - L_\pi(\pi')| \leq \frac{2C\gamma}{(1-\gamma)^2} D_{\text{KL}}^{\max}(\pi, \pi')$$

△Hints

1. Apply the Cauchy-Schwarz inequality and results from the previous exercise.
2. Apply Pinsker's inequality: $D_{\text{TV}}^{\max}(\pi, \pi') \leq \sqrt{\frac{1}{2} D_{\text{KL}}^{\max}(\pi, \pi')}$.

Although $\delta(\pi, \pi') = \frac{2C\gamma}{(1-\gamma)^2} D_{\text{KL}}^{\max}(\pi, \pi')$, this value is often practically very small, leading to many iterations to achieve significant policy changes. To address this, δ is treated instead as a hyperparameter. The optimization problem for π' can then be written as:

$$\max_{\pi'} L_\pi(\pi') = \max_{\pi'} \sum_s d^\pi(s) \sum_a \pi'(a|s) A_\pi(s, a) \quad \text{s.t. } D_{\text{KL}}^{\max}(\pi, \pi') \leq \delta$$

We introduced $L_\pi(\pi')$ instead of $J(\pi')$ because states are sampled according to the old policy $s_t \sim d^\pi$, not the future policy $s_t \sim d^{\pi'}$. Similarly, actions are not sampled according to π' , so the optimization problem can be further rewritten using importance sampling:

$$\max_{\pi'} \sum_s d^\pi(s) \sum_a \pi(a|s) \left[\frac{\pi'(a|s)}{\pi(a|s)} A_\pi(s, a) \right] = \max_{\pi'} \mathbb{E}_{s \sim d^\pi, a \sim \pi(\cdot|s)} \left[\frac{\pi'(a|s)}{\pi(a|s)} A_\pi(s, a) \right]$$

Furthermore, to simplify the constraint, we replace the max-KL constraint with the expected-KL:

$$\mathbb{E}_{s \sim d^\pi} [D_{\text{KL}}(\pi(\cdot|s), \pi'(\cdot|s))] \leq \delta.$$

This approximation effectively handles both the objective and the constraint and is the final optimization problem that TRPO solves.

References

- [1] Sudharsan Ravichandiran. *Deep Reinforcement Learning with Python: Master classic RL, deep RL, distributional RL, inverse RL, and more with OpenAI Gym and TensorFlow*. Packt Publishing Ltd, 2020.
- [2] OpenAI. Introduction to reinforcement learning - spinning up documentation, https://spinningup.openai.com/en/latest/spinningup/rl_intro3.html#id7.
- [3] John Schulman. Trust region policy optimization. *arXiv preprint arXiv:1502.05477*, 2015.